

Q.1)

```
CREATE DATABASE EMP;
```

```
USE EMP;
```

```
CREATE TABLE Employee (
```

```
    EMP_ID INT PRIMARY KEY,
```

```
    EMP_NAME VARCHAR(50),
```

```
    SALARY DECIMAL(10, 2),
```

```
    DEPARTMENT VARCHAR(50)
```

```
);
```

```
INSERT INTO Employee (EMP_ID, EMP_NAME, SALARY, DEPARTMENT) VALUES
```

```
(101, 'Alice Smith', 95000.00, 'HR'),
```

```
(102, 'Bob Jones', 80000.00, 'HR'),
```

```
(103, 'Charlie Brown', 120000.00, 'IT'),
```

```
(104, 'David Miller', 120000.00, 'IT'),
```

```
(105, 'Emma Davis', 95000.00, 'IT'),
```

```
(106, 'Frank Wilson', 85000.00, 'IT'),
```

```
(107, 'Grace Lee', 110000.00, 'Finance'),
```

```
(108, 'Henry Clark', 110000.00, 'Finance'),
```

```
(109, 'Ivy Wright', 90000.00, 'Finance'),
```

```
(110, 'Jack Turner', 75000.00, 'Finance');
```

MySQL Workbench

1-JUL* Administration - Server Status SQL File 2 SQL File 3

File Edit View Query Database Server Tools Scripting Help

Navigation: 1-JUL* Administration - Server Status SQL File 2 SQL File 3

MANAGEMENT

- Server Status
- Client Connections
- Users and Privilege
- Status and System
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Administration

Information

Connection Details

Name: 127.0.0.1 3306

root

root@localhost

TLS_AES_128_GCM

Server

MySQL Community

Server - GPL

8.0.45

Connector

C++ 9.6.0

1

```
CREATE DATABASE EMP;
```

2

```
USE EMP;
```

3

```
CREATE TABLE Employee (
```

4

```
    EMP_ID INT PRIMARY KEY,
```

5

```
    EMP_NAME VARCHAR(50),
```

6

```
    SALARY DECIMAL(10, 2),
```

7

```
    DEPARTMENT VARCHAR(50)
```

8

```
);
```

9

10

```
INSERT INTO Employee (EMP_ID, EMP_NAME, SALARY, DEPARTMENT) VALUES
```

11

```
(101, 'Alice Smith', 95000.00, 'HR'),
```

12

```
(102, 'Bob Jones', 80000.00, 'HR'),
```

13

```
(103, 'Charlie Brown', 120000.00, 'IT'),
```

14

```
(104, 'David Miller', 120000.00, 'IT'),
```

15

```
(105, 'Emma Davis', 95000.00, 'IT');
```

Result Grid

EMP_ID	EMP_NAME	SALARY	DEPARTMENT
101	Alice Smith	95000.00	HR
102	Bob Jones	80000.00	HR
103	Charlie Brown	120000.00	IT
104	David Miller	120000.00	IT
105	Emma Davis	95000.00	IT

Employee 34 x

Apply Revert Context Help Snippets

Output

#	Time	Action	Message	Duration / Fetch
80	11:58:46	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT OR...	10 row(s) returned	0.016 sec / 0.000 sec
81	11:58:53	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT OR...	10 row(s) returned	0.000 sec / 0.000 sec
82	11:59:12	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MAX(SALARY) OVER(PARTITION BY DEPARTMENT OR...	10 row(s) returned	0.000 sec / 0.000 sec
83	11:59:37	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
84	11:59:53	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
85	12:00:51	SELECT * FROM Employee LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Insights:

We create Employee table and insert data in it.

Q.2)

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
  
ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
  
FROM Employee;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a query that selects employee details and assigns a row number based on salary. The Results tab displays the output of the query, showing columns EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and SALARY_RANK. The Output tab shows the execution log with timestamps and messages for each row returned.

SQL Query:

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;
```

Result Grid:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
103	Charlie Brown	120000.00	IT	1
104	David Miller	120000.00	IT	2
107	Grace Lee	110000.00	Finance	3
108	Henry Clark	110000.00	Finance	4
101	Alice Smith	95000.00	HR	5
102	Grace Smith	95000.00	IT	6

Output Log:

#	Time	Action	Message	Duration / Fetch
70	11:50:40	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY+(SALARY)) OVER(PARTITION BY DEPAR...	10 row(s) returned	0.000 sec / 0.000 sec
71	11:51:14	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
72	11:54:29	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
73	11:55:43	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, FIRST VALUE (SALARY) OVER(PARTITION BY DEPART...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server ve...	0.000 sec
74	11:56:08	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
75	11:56:42	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

This query adds new column and gives row numbers.

Q.3)

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
RANK() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a query that assigns ranks to employees based on their salary. The Results window displays the output of this query, showing employee details and their corresponding rank. The Output window shows the execution log of the query.

SQL Query:

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
RANK() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;
```

Query Results:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
103	Charlie Brown	120000.00	IT	1
104	David Miller	120000.00	IT	1
107	Grace Lee	110000.00	Finance	3
108	Henry Clark	110000.00	Finance	3
101	Alice Smith	95000.00	HR	5
102	Grace Davis	85000.00	IT	6

Output Log:

#	Time	Action	Message	Duration / Fetch
71	11:51:14	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
72	11:54:29	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
73	11:55:43	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, FIRST VALUE (SALARY) OVER(PARTITION BY DEPART...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server ve...	0.000 sec
74	11:56:08	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
75	11:56:42	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
76	11:57:05	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, RANK() OVER(ORDER BY SALARY DESC) AS SALARY...	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

It gives rank according to employee Salary but it skips some values .

Q.4)

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
  
DENSE_RANK() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
  
FROM Employee;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
RANK() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
DENSE_RANK() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;
```

The Results Grid shows the output of the first query, displaying columns EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and SALARY_RANK. The results are as follows:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
103	Charlie Brown	120000.00	IT	1
104	David Miller	120000.00	IT	1
107	Grace Lee	110000.00	Finance	2
108	Henry Clark	110000.00	Finance	2
101	Alice Smith	95000.00	HR	3
106	Robert Prince	90000.00	HR	4

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
72	11:54:29	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) A...	10 row(s) returned	0.000 sec / 0.000 sec
73	11:55:43	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, FIRST VALUE (SALARY) OVER(PARTITION BY DEPART...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server ve...	0.000 sec
74	11:56:08	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
75	11:56:42	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
76	11:57:05	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, RANK() OVER(ORDER BY SALARY DESC) AS SALARY...	10 row(s) returned	0.000 sec / 0.000 sec
77	11:57:34	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(ORDER BY SALARY DESC) AS S...	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

It gives Rank in ascending order with some repetition without skipping any rank .

Q.5)

SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,

DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS
SALARY_RANK

FROM Employee;

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query with syntax errors. The results pane shows a table with 5 columns: EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and SALARY_RANK. The data is as follows:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
107	Grace Lee	110000.00	Finance	1
108	Henry Clark	110000.00	Finance	1
109	Ivy Wright	90000.00	Finance	2
110	Jack Turner	75000.00	Finance	3
101	Alice Smith	95000.00	HR	1

The output pane shows the execution of the query, with a message indicating an error: "Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version...".

Insights:

In this query we see Department wise salary Rank.

Q.6)

SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,

SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS
SALARY_RANK

FROM Employee;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
DENSE_RANK() OVER(ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY ) AS SALARY_RANK
```

The Results tab shows the output of the first query, displaying columns EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and SALARY_RANK. The data is as follows:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
107	Grace Lee	110000.00	Finance	220000.00
108	Henry Clark	110000.00	Finance	220000.00
109	Ivy Wright	90000.00	Finance	310000.00
110	Jack Turner	75000.00	Finance	385000.00
101	Alice Smith	95000.00	HR	95000.00

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
74	11:56:08	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
75	11:56:42	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, ROW_NUMBER() OVER(ORDER BY SALARY DESC) AS ...	10 row(s) returned	0.000 sec / 0.000 sec
76	11:57:05	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, RANK() OVER(ORDER BY SALARY DESC) AS SALARY...	10 row(s) returned	0.000 sec / 0.000 sec
77	11:57:34	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(ORDER BY SALARY DESC) AS S...	10 row(s) returned	0.000 sec / 0.000 sec
78	11:58:00	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(PARTITION BY DEPARTMENT O...	10 row(s) returned	0.000 sec / 0.000 sec
79	11:58:21	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, SUM(SALARY) OVER(PARTITION BY DEPARTMENT OR...	10 row(s) returned	0.016 sec / 0.000 sec

Insights:

This query adds The all department people salary one by one.

Q.7)

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY ) AS SALARY_RANK  
FROM Employee;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
35 DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
36 FROM Employee;  
37  
38 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
39 SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
40 FROM Employee;  
41  
42 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
43 MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY ) AS SALARY_RANK  
44 FROM Employee;  
45  
46 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
47 MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
48 FROM Employee;  
49
```

The Results grid shows the output of the query, displaying columns EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and SALARY_RANK. The data is as follows:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
110	Jack Turner	75000.00	Finance	75000.00
109	Ivy Wright	90000.00	Finance	75000.00
107	Grace Lee	110000.00	Finance	75000.00
108	Henry Clark	110000.00	Finance	75000.00
102	Bob Jones	80000.00	HR	80000.00
101	Alice Smith	90000.00	HR	80000.00

The Output tab shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
76	11:57:05	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK	10 row(s) returned	0.000 sec / 0.000 sec
77	11:57:34	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK	10 row(s) returned	0.000 sec / 0.000 sec
78	11:58:00	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK	10 row(s) returned	0.000 sec / 0.000 sec
79	11:58:21	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK	10 row(s) returned	0.016 sec / 0.000 sec
80	11:58:46	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY) AS SALARY_RANK	10 row(s) returned	0.016 sec / 0.000 sec
81	11:58:53	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY) AS SALARY_RANK	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

It gives the minimum salary department wise .

Q.8)

SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,

MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS
SALARY_RANK

FROM Employee;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
35 DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK
36 FROM Employee;
37
38 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,
39 SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK
40 FROM Employee;
41
42 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,
43 MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY ) AS SALARY_RANK
44 FROM Employee;
45
46 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,
47 MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK
48 FROM Employee;
49
```

The Results window displays the output of the last query (line 47-48). The table has 5 columns: EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and SALARY_RANK. The data is as follows:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	SALARY_RANK
107	Grace Lee	110000.00	Finance	110000.00
108	Henry Clark	110000.00	Finance	110000.00
109	Ivy Wright	90000.00	Finance	110000.00
110	Jack Turner	75000.00	Finance	110000.00
101	Alice Smith	95000.00	HR	95000.00

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
77	11:57:34	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS S...	10 row(s) returned	0.000 sec / 0.000 sec
78	11:58:00	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS S...	10 row(s) returned	0.000 sec / 0.000 sec
79	11:58:21	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS S...	10 row(s) returned	0.016 sec / 0.000 sec
80	11:58:46	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY) AS S...	10 row(s) returned	0.016 sec / 0.000 sec
81	11:58:53	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY) AS S...	10 row(s) returned	0.000 sec / 0.000 sec
82	11:59:12	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS S...	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

MAX gives the maximum salary per department in order.

Q.9)

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS AVG_SALARY  
FROM Employee;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a query that calculates the average salary per department using a window function. The query is as follows:

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY ) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS AVG_SALARY  
FROM Employee;  
  
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS AVG_SALARY,
```

The Results tab shows the output of the query. The columns are EMP_ID, EMP_NAME, SALARY, DEPARTMENT, and AVG_SALARY. The data is as follows:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	AVG_SALARY
107	Grace Lee	110000.00	Finance	96250.000000
108	Henry Clark	110000.00	Finance	96250.000000
109	Ivy Wright	90000.00	Finance	96250.000000
110	Jack Turner	75000.00	Finance	96250.000000
101	Alice Smith	95000.00	HR	87500.000000

The Output tab shows the execution of the query. The columns are #, Time, Action, Message, and Duration / Fetch. The data is as follows:

#	Time	Action	Message	Duration / Fetch
78	11:58:00	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, DENSE_RANK() OVER(PARTITION BY DEPARTMENT ORDER BY SALARY) AS SALARY_RANK	10 row(s) returned	0.000 sec / 0.000 sec
79	11:58:21	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, SUM(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK	10 row(s) returned	0.016 sec / 0.000 sec
80	11:58:46	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY) AS SALARY_RANK	10 row(s) returned	0.016 sec / 0.000 sec
81	11:58:53	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK	10 row(s) returned	0.000 sec / 0.000 sec
82	11:59:12	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) AS AVG_SALARY	10 row(s) returned	0.000 sec / 0.000 sec
83	11:59:37	SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT, AVG(SALARY) OVER(PARTITION BY DEPARTMENT) AS AVG_SALARY,	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

It gives the average salary of per department.

Q.10)

```
SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS AVG_SALARY,  
SALARY- AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS DIFF  
FROM Employee;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
5 Execute the selected portion of the script or everything, if there is no selection  
43 MIN(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY ) AS SALARY_RANK  
44 FROM Employee;  
45  
46 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
47 MAX(SALARY) OVER(PARTITION BY DEPARTMENT ORDER BY SALARY DESC) AS SALARY_RANK  
48 FROM Employee;  
49  
50 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
51 AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS AVG_SALARY  
52 FROM Employee;  
53  
54 SELECT EMP_ID,EMP_NAME,SALARY,DEPARTMENT,  
55 AVG(SALARY) OVER(PARTITION BY DEPARTMENT ) AS AVG_SALARY,
```

The Results grid shows the following data:

EMP_ID	EMP_NAME	SALARY	DEPARTMENT	AVG_SALARY	DIFF
107	Grace Lee	110000.00	Finance	96250.000000	13750.000000
108	Henry Clark	110000.00	Finance	96250.000000	13750.000000
109	Ivy Wright	90000.00	Finance	96250.000000	-6250.000000
110	Jack Turner	75000.00	Finance	96250.000000	-21250.000000
101	Alice Smith	95000.00	HR	87500.000000	7500.000000

The Output tab shows the execution of the query, with messages indicating that the query was executed successfully and the results were returned.

Insights:

It shows the difference between average salary and employee salary.